

Week 3: Spring Data JPA with Spring Boot, Hibernate

Hands on 4

Difference between JPA, Hibernate and Spring Data JPA

Sol.:

Java Persistence API (JPA)

JPA, Hibernate and Spring Data JPA are all useful tools for working with databases in Java, but they each do slightly different things. JPA is not a framework itself, but it's more like a rule book – a specification that tells how Java objects should be mapped to database tables. It doesn't do anything on its own, so it needs something like Hibernate to actually work. So if you are just using JPA alone, you won't be able to save data unless you have a provider.

Hibernate

Hibernate is one of the most popular implementations of JPA, and it also adds some extra features that go beyond the JPA standard. With Hibernate, you have to manage the session and transactions yourself. That means more boilerplate code and some more complexity, but it also gives more control to the developer over how things work under the hood. Sometimes that can be an advantage.

Spring Data JPA

Spring Data JPA makes things even easier by adding a layer on top of JPA and Hibernate. It lets you create repository interfaces and you don't even have to write the SQL or JPQL queries for common tasks. Spring takes care of a lot of the hard work like transactions, query generation and session handling. This makes it ideal for quick development when you're building a Spring Boot application and want things to just work out of the box.

Example -

1. Hibernate Example

Requires manual session and transaction management.

```
public Integer addEmployee(Employee employee) {  
  
    Session session = factory.openSession();
```

```

Transaction tx = null;

Integer employeeID = null;

try {

    tx = session.beginTransaction();

    employeeID = (Integer) session.save(employee);

    tx.commit();

} catch (HibernateException e) {

    if (tx != null) tx.rollback();

    e.printStackTrace();

} finally {

    session.close();

}

return employeeID;

}

```

2. Spring Data JPA Example

Clean, minimal, and powerful — no boilerplate required.

EmployeeRepository.java

```

public interface EmployeeRepository extends JpaRepository<Employee,
Integer> {

}

```

EmployeeService.java

```

@Service

```

```
public class EmployeeService {  
  
    @Autowired  
  
    private EmployeeRepository employeeRepository;  
  
    @Transactional  
  
    public void addEmployee(Employee employee) {  
  
        employeeRepository.save(employee);  
  
    }  
  
}
```